



**Coláiste na Tríonóide, Baile Átha Cliath**  
**Trinity College Dublin**

Ollscoil Átha Cliath | The University of Dublin

**Faculty of Science, Technology, Engineering and Mathematics**

**School of Computer Science & Statistics**

**Senior Freshman**  
**Computer Science and Language**  
**Computer Science and Business**

**Semester 2 2025**

**2062 - Natural Language Processing**

**Friday 25th April 2025**

**17:00 – 19:00**

**Dr. Martin Emms**

**Instructions to Candidates:**

There are **4** questions. Answer any **2**.

Each question is worth 50 marks.

There is a **Supplementary Info** sheet giving some information required for particular questions.

# 1. Question concerns top-down stack-based parser

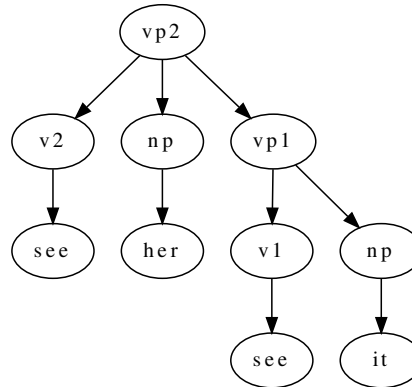
(a) let G1 be the grammar to the left below

```

initial(vp2)
vp1 --> v1,np
v1 --> [see]
v1 --> [touch]
vp2 --> v2,np,vp1
v2 --> [see]
v2 --> [let]

np --> [her]
np --> [it]

```



i. The tree via which 'see her see it' is categorised as vp2 according to G1 is shown to the right above. The standard top-down parser develops a tree by left-most expansion. Show the evolution of the above tree, as developed by the standard top-down parser, starting from a single node vp2 tree. [5 marks]

ii. The *simple* top-down parser (without backtracking) will accept 'see her see it'. Give the sequence of parser configurations, tracing the execution of this parser. [4 marks]

tested with standard course td\_tester

(b) Let G2 be the grammar that results by augmenting G1 by the 3 rules below

```

np --> det,n
det --> [her]
n --> [touch]

```

and in particular situate these rules *earlier* than the np rules already present in G1.

i. Assuming G2, the *simple* top-down parser (without backtracking) will reject 'see her touch it'. Give the sequence of parser configurations, tracing the execution of the parser rejecting this input. [4 marks]

ii. Assuming G2, the backtracking version of the top-down parser will accept 'see her touch it'. Give the sequence of parser configurations, tracing the execution of the parser accepting this input (including some commentary on backtracking). [7 marks]

- (c) The following classes could be used in implementing such a top-down, backtracking parser (as well as the CFG classes shown in the **Supplementary Info**):

|                                                                                                                        |                                                                                                                                                                                                                                                                                                                    |
|------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre>class Chce { public:     Chce();     vector&lt;Category&gt; stack;     int rule_index;     int progress; };</pre> | <pre>class PARSER { public:     bool accept(vector&lt;string&gt; input); private:     bool final_config()     int move();     Grammar g;     vector&lt;string&gt; input;     int progress; // how far thru input     int rule_index;     vector&lt;Category&gt; stack;     vector&lt;Chce&gt; chce_stack; };</pre> |
|------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

Figure 1 (next page) gives partial code for the `PARSER::accept` method. For the numbered commented locations give the necessary code. In doing so, assume the `PARSER::move` method *classifies* whether a move can be made using a Rule whose index is  $\geq$  `rule_index`, with 1 indicating if leaf cancellation is possible, 2 indicating left expansion, and 0 if no move is possible. In cases 1 and 2, `rule_index` is set to the index of the relevant rule.

Assume that terminal category [smthg] only ever appears in rules of the form  
`x --> [smthg]`

Assume every move made creates a choice-point.

[20 marks]

- (d) The top-down, backtracking algorithm has a non-termination problem in the case of grammars that have *left recursive* rules. Give a grammar including left-recursion and show with a trace of the successive configurations of the parser how non-termination arises.

[10 marks]

```

bool PARSER::accept(vector<string> key) {

    bool backtracking = true;
    //1. initialize stack, chce_stack, rule_index, progress, input

    do {
        while((move_type = move()) != 0) {
            if(move_type == 1) {
                //2. update chce_stack, stack, progress;
                rule_index=0;
            }
            else if(move_type == 2) {
                //3. update chce_stack, stack, progress;
                rule_index = 0;
            }
        }

        if (final_config()) {
            accept = true;
            break;
        }
        else if (choice_stk.size() != 0) {
            //4. set stack, progress, rule_index from chce_stack;
            //5. update chce_stack;
        }
        else {
            backtracking = false;
            accept = false;
        }
    } while(backtracking);

    return accept;
}

```

Figure 1: Outline of top-down backtracking algorithm

## 2. Question concerns chart-parsers

(a) let  $\mathcal{G}_1$  be the grammar:

|               |                |
|---------------|----------------|
| initial(vp)   | va --> [order] |
| vp --> L,R    | vb --> [parse] |
| L --> va,np   | np --> [it]    |
| R --> [to],vp | np --> [him]   |
| vp --> vb,np  |                |

- The input 'order him to parse it' is of category vp according to  $\mathcal{G}_1$ . Give its analysis tree. [2 marks]
- Give the chart which would be made by the standard CKY parser on this. [8 marks]

(b) Assume the following outline of a CHART class (and the CFG classes from the **Supplementary Info**):

```
class CHART {
public:
    Grammar g;
    vector<string> words; // sequence to parse

    /* a member 'chart' such that chart[i][k] is the chart's content
       concerning start = i and length = k */

    bool accept(vector<string> input);
    void initialize(); // set chart dimensions then put in length 1 entries
    void do_cell(int i, int k); // fill in all entries at chart[i][k]
    bool is_there(int i, int k, Category c); // check for c at chart[i][k]
    void add_edge(int i, int k, Category c); // add c to chart[i][k] if not there
};
```

- Assume that `CHART::chart` encodes somehow a 2-dimensional *start-vs-length* table, each slot of which represents a collection of Category objects.

Assume `accept(..)` calls `initialize()` to add all length 1 entries and then `do_cell(i,k)` is repeatedly called in such a way that, for all possible  $(i, k)$  pairings of start and length, every category  $C$  goes into `chart[i][k]` which is implied by rules applied to existing entries for shorter lengths  $k'$  at appropriate positions.

Assuming `CHART::is_there(..)` and `CHART::add_edge(..)` are already implemented, give an implementation of `do_cell(..)`. [15 marks]

(question continues next page)

- ii. Now assume the underspecified `CHART::chart` member is actually declared

```
vector<vector<vector<Category>>> chart;
```

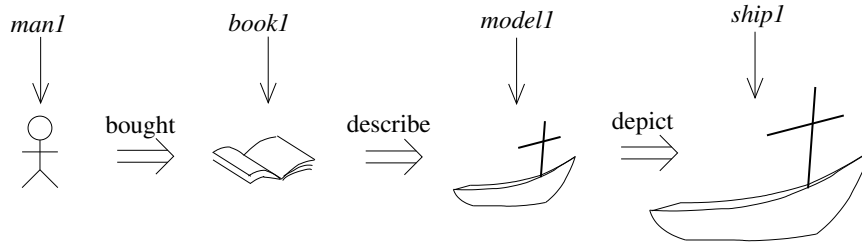
Give implementations of `CHART::is_there(...)` and `CHART::add(...)`. [7 marks]

- (c) Suppose  $\mathcal{G}_2$  is the grammar that derives from  $\mathcal{G}_1$  by replacing the 3 rules  $(\alpha)$  below, with the single rule  $(\beta)$  below.

$(\alpha)$   $vp \rightarrow L, R$        $L \rightarrow va, np$        $R \rightarrow [to], vp$        $(\beta)$   $vp \rightarrow va, np, [to], vp$

- i. State the aspect of  $\mathcal{G}_2$  which makes it not suited to the standard CKY parser [2 marks]
- ii. There is an adaptation of the CKY algorithm using '*partial constituents*', with categories of the form  $C/d_i \dots d_n$ , which makes it suited to a grammar such as  $\mathcal{G}_2$ . Assuming this adaptation give the chart that will be produced on the input '*order him to parse it*'. [12 marks]
- (d) In both variants of the CKY algorithm, `chart[i][k]` stores all the possible categories of sequences starting at  $i$  and of length  $k$ , and the algorithm has better than exponential running time. Briefly explain why this could not be so if the cells stored not all *categories* but instead all the *trees* for those categories. [4 marks]

### 3. Question concerns Centre-embedding and Slash-grammars



- (a) Referring to the above picture, *ship1* might be identified via its relation to *model1* as in (i) below, and *model1* via its relation to *book1* as in (ii) below.

(i) *the ship that model1 depicted*    (ii) *the model that book1 described*

When (ii)'s description of *model1* is substituted into (i) the result is a noun-phrase with an object relative clause which contains a further such noun-phrase. Give this noun-phrase [2 marks]

- (b) The process could be continued, for example replacing *book1* with *the book that man1 bought*. Let  $\mathcal{NP\_OR}$  stand for the language obtained by repeatedly doing this, with contents defined as follows:

$$\begin{aligned}\mathcal{NP\_OR} &= \{(\text{the } N \text{ that})^n \text{ PN } (\text{TV})^n \mid n \geq 1\} \text{ where} \\ N &\in \{\text{ship}, \text{model}, \text{book}, \text{man}\} \quad \text{TV} \in \{\text{depicted}, \text{described}, \text{bought}\} \\ \text{PN} &= \{\text{ship1}, \text{model1}, \text{book1}, \text{man1}\}\end{aligned}$$

- i. Give a CFG which also defines the set  $\mathcal{NP\_OR}$ . [6 marks]
  - ii.  $\mathcal{NP\_OR}$  is very analogous to the language  $\mathcal{P} = \{a^n b^n \mid n \geq 1\}$ . Outline the proof using the Pumping Lemma (see **Supplementary Info**) which shows that  $\mathcal{P}$  *could not* be defined by an FSA [8 marks]
- (c) Suppose the language  $\mathcal{NP\_SR}$  is defined as follows:

$$\begin{aligned}\mathcal{NP\_SR} &= \{(\text{the } N \text{ that TV})^n \text{ PN} \mid n \geq 1\} \text{ where} \\ N, \text{ TV}, \text{ PN} &\text{ are as for } \mathcal{NP\_OR}\end{aligned}$$

- i. Give a CFG which also defines the set  $\mathcal{NP\_SR}$ . [6 marks]
- ii. Give also an FSA which defines  $\mathcal{NP\_SR}$ . [3 marks]

(question continues next page)

- (d) The fundamental fact about relative clauses is that what follows the relative pronoun should be

*any sentence from which some noun-phrase has been removed* (Rel Gen)

'slash grammars' deal with this by deriving rules involving 'slash categories' from rules which do not, such that  $s/np$  categorises the strings that fit **Rel Gen** above.

Let  $\mathcal{G}_1$  be the set of rules

|                                    |                         |                           |                              |
|------------------------------------|-------------------------|---------------------------|------------------------------|
| $s \rightarrow np, vp$             | $det \rightarrow [the]$ | $np \rightarrow [man1]$   | $tv \rightarrow [bought]$    |
| $np \rightarrow det, n$            | $n \rightarrow [man]$   | $np \rightarrow [book1]$  | $tv \rightarrow [described]$ |
| $np \rightarrow pn$                | $n \rightarrow [book]$  | $np \rightarrow [model1]$ | $tv \rightarrow [depicted]$  |
| $vp \rightarrow tv, np$            | $n \rightarrow [model]$ | $np \rightarrow [ship1]$  | $dtv \rightarrow [showed]$   |
| $vp \rightarrow dtv, np, [to], np$ | $n \rightarrow [ship]$  | $np \rightarrow [Ernie]$  |                              |
|                                    |                         | $np \rightarrow [Bert]$   |                              |

and  $\mathcal{G}_2$  be the singleton set  $\{ np \rightarrow det, n, [that], s/np \}$ .

Extend  $\mathcal{G}_1 \cup \mathcal{G}_2$  to a larger CFG allowing for nps with relative clauses, based on the **Rel Gen** generalisation, by adding suitable rules involving  $s/np$  (and others of the form  $X/np$ ).

Ensure as you do so that you achieve coverage of the following, drawing the trees according to your grammar to verify this

*the ship that model1 depicted*

*the model that depicted ship1*

*the man that Bert showed the book to*

[17 marks]

- (e)  $\mathcal{G}_1$  lacks verbs which allow an  $s$  to be a sub-constituent of a larger  $s$ , as in

Bert believed *model1 depicted ship1*

Ernie said *Bert believed model1 depicted ship1*

Add rules to allow these and the further related nps with relative clauses such as:

*the ship that Bert believed model1 depicted*

*the ship that Ernie said Bert believed model1 depicted*

[5 marks]

- (f) Draw the analysis tree according to your grammar of

*the ship that Bert believed model1 depicted*

[3 marks]



#### 4. Question concerns computing a semantic value

Note: refer to the **Semantics** section in the **Supplementary Info**

(a) Suppose  $p$  is a Rel object to represent arity 1 relations. What will the contents of  $p$  be on executing the following:

i.  $p = \text{reduce}(\text{likes}, 1, \text{"maths"})$ ; [2 marks]

ii.  $p = \text{reduce}(\text{likes}, 0, \text{"Faraday"})$ ; [2 marks]

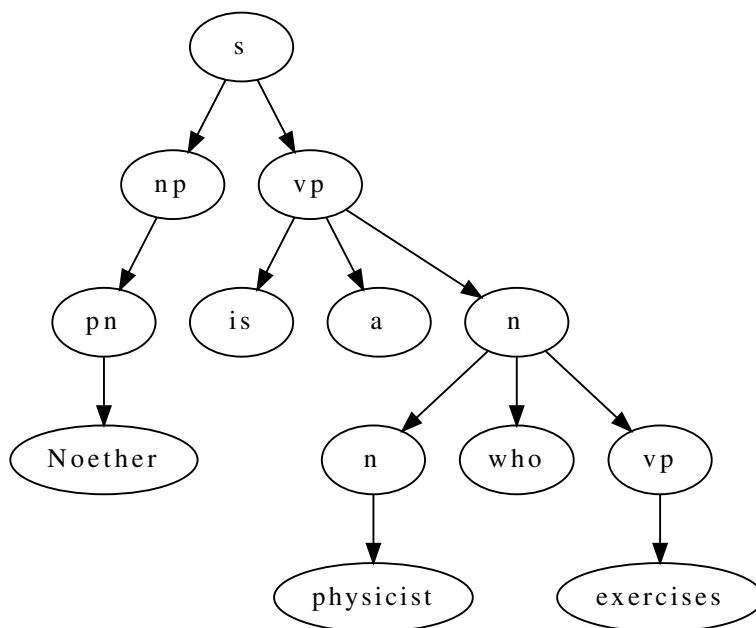
(b) Provide code implementing  $\text{Rel reduce}(\text{Rel } r, \text{int } which, \text{Thing } v)$ , using the helper function  $\text{Tuple erase}(\text{Tuple } t, \text{int } i)$  [8 marks]

(c) Give implementations of the following functions

i.  $\text{bool every}(\text{Rel } P, \text{Rel } Q)$ : returns true when every tuple  $x$  which is in  $P$ 's tuples is in  $Q$ 's tuples also [2 marks]

ii.  $\text{Rel meet}(\text{Rel } P, \text{Rel } Q)$ : given arity-1 Rel objects  $P$  and  $Q$  it returns the *intersection* of these. [4 marks]

(d) Consider computing a semantic value for the tree



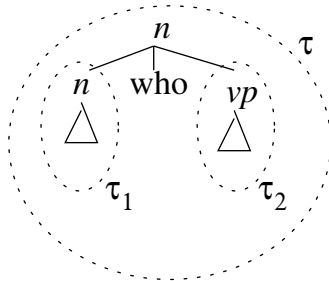
Suppose the case of  $\text{Rel compute\_vp\_value}(\text{Tree } *t)$  suited to the upper  $vp$  node equates the  $vp$ 's value with that of its  $n$  part.

(question continues next page)

Concerning other parts of the tree, complete the parts enclosed in `/* .. */` in the partial semantic evaluation rules below:

(i) suited to the uppermost *n* node

[10 marks]



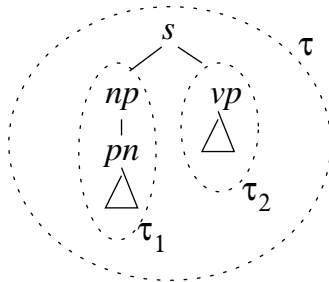
```

Rel compute_n_value(Tree *τ) {
  if(τ, τ₁, τ₂ are as shown to the left) {
    Rel r(1);
    /* code using compute_vp_value(..)
       compute_n_value(..) */
    return r;
  }
  else { poss other cases}
}

```

(ii) suited to the *s* node

[10 marks]



```

bool compute_s_value(Tree *τ) {
  bool v;
  if(τ, τ₁, τ₂ are as shown to the left) {
    /* code using compute_np_value(..)
       compute_vp_value(..) */
    return v;
  }
  else { poss other cases}
}

```

(e) Consider

( $\alpha$ ) *Noether is a physicist who exercises*

( $\beta$ ) *Noether is a physicist*

( $\gamma$ ) *Noether exercises*

No matter what the settings of the relations, the semantic evaluation functions are such that it will never be the case that ( $\alpha$ ) gets value true whilst either of ( $\beta$ ) or ( $\gamma$ ) gets value false. Show that this is the case.

[12 marks]